

Unit 2 – Array-based Lists

CS 201 - Data Structures II

Spring 2024

Habib University

Syeda Saleha Raza

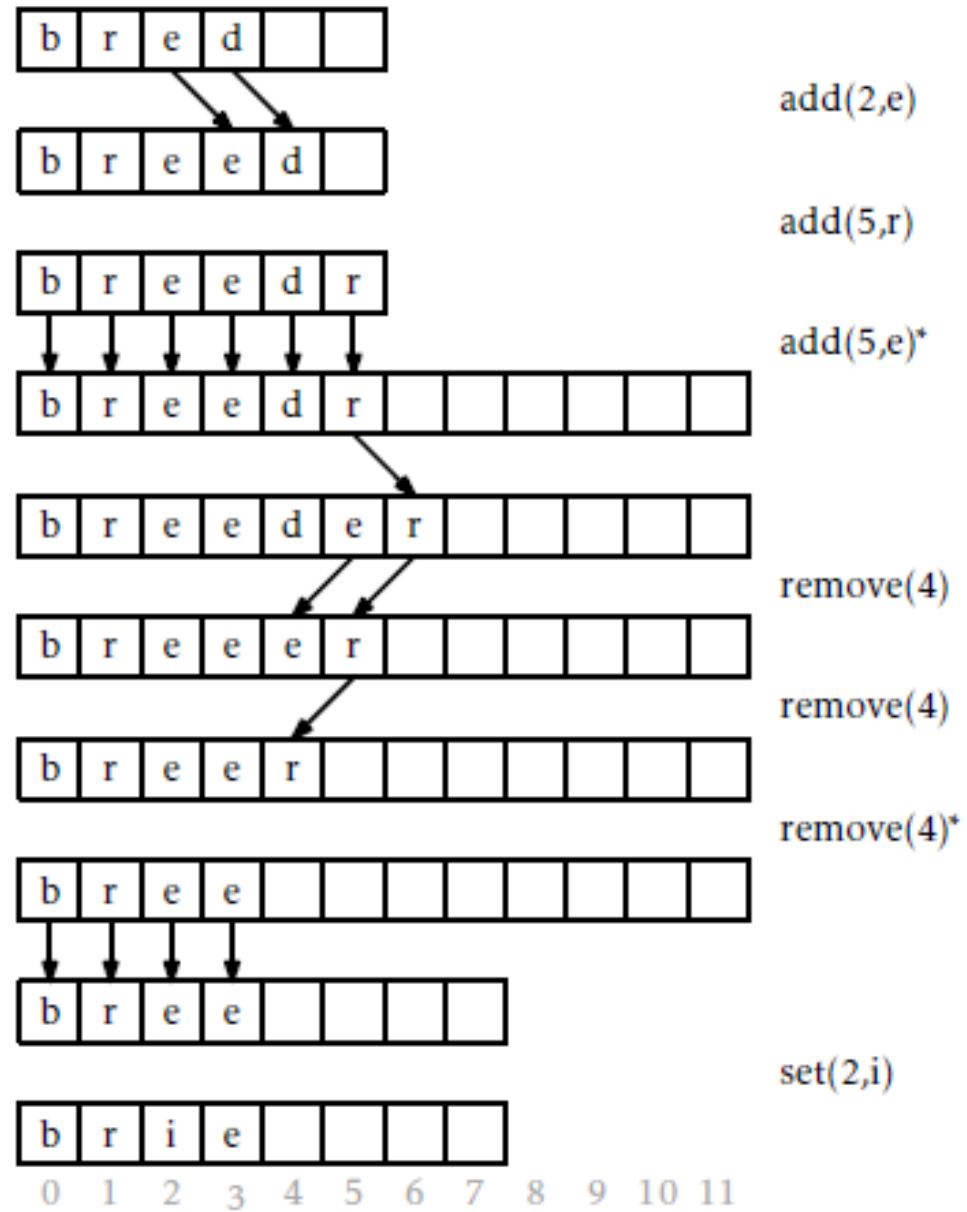
Arrays

- Arrays offer constant time access to any value in the array. This is what allows *get(i)* and *set(i;x)* to run in constant time.
- Arrays are not very dynamic. Adding or removing an element near the middle of a list means that a large number of elements in the array need to be shifted to make room for the newly added element or to fill in the gap created by the deleted element. This is why the operations *add(i; x)* and *remove(i)* have running times that depend on n and i .
- Arrays cannot expand or shrink.

Backing Array

- Array that resizes itself dynamically
- Resizing an array of size n means that it allocates a new array b whose size is $2n$ and copies the n elements of a into the first n positions in b , and then sets a to b . Thus, after a call to `resize()`, $\text{length}(a) = 2n$.

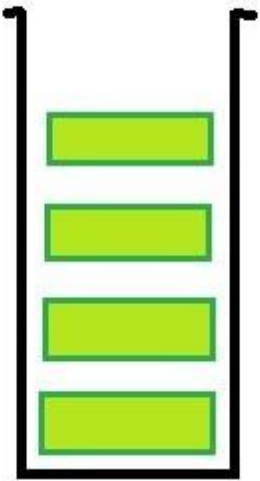
```
resize()  
   $b \leftarrow \text{new\_array}(\max(1, 2 \cdot n))$   
   $b[0, 1, \dots, n - 1] \leftarrow a[0, 1, \dots, n - 1]$   
   $a \leftarrow b$ 
```



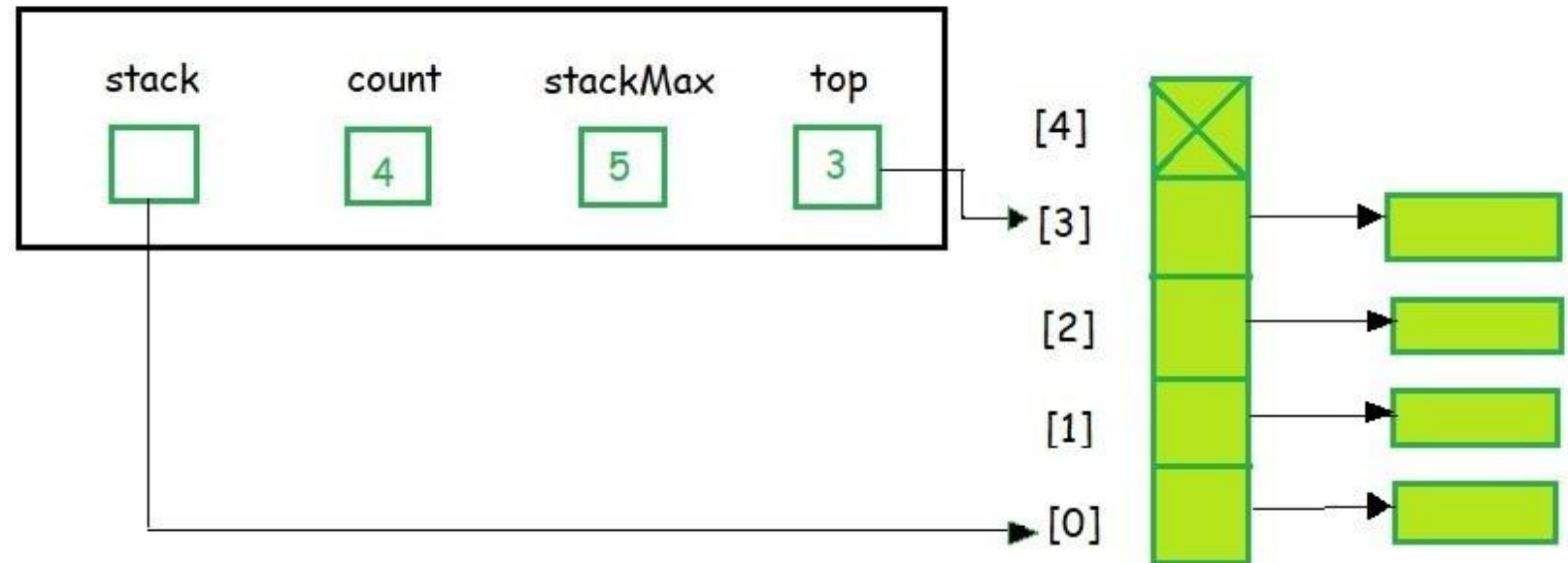
ArrayStack

Stack ADT

a) Conceptual



b) Physical Structure



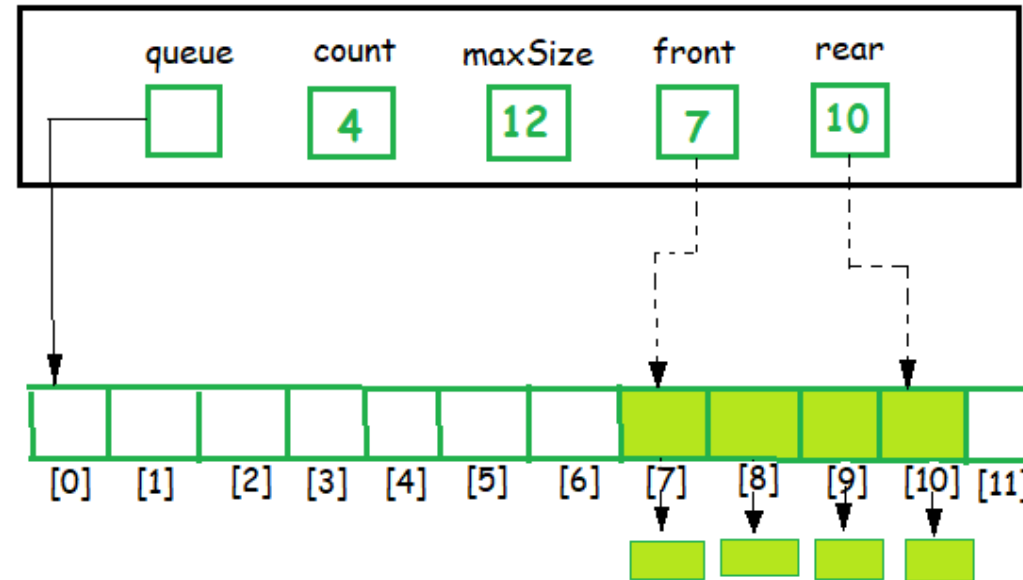
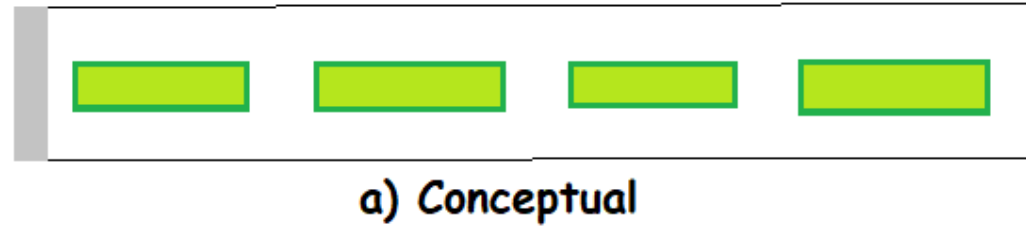
<https://www.geeksforgeeks.org/abstract-data-types>

ArrayStack

- Time Complexity of Push and Pop operations
- Using memcpy operations

ArrayQueue

Queue ADT

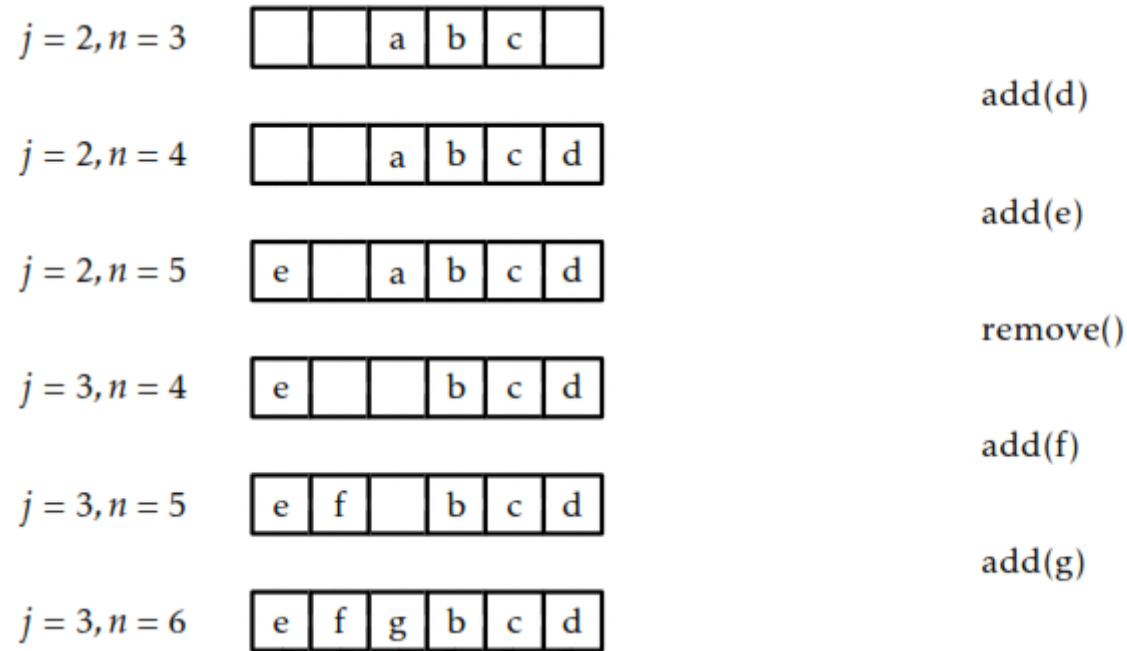


b) Physical Structures

<https://www.geeksforgeeks.org/abstract-data-types>

ArrayQueue

- Time complexity of enqueue and deque operations



ArrayQueue – Making it circular

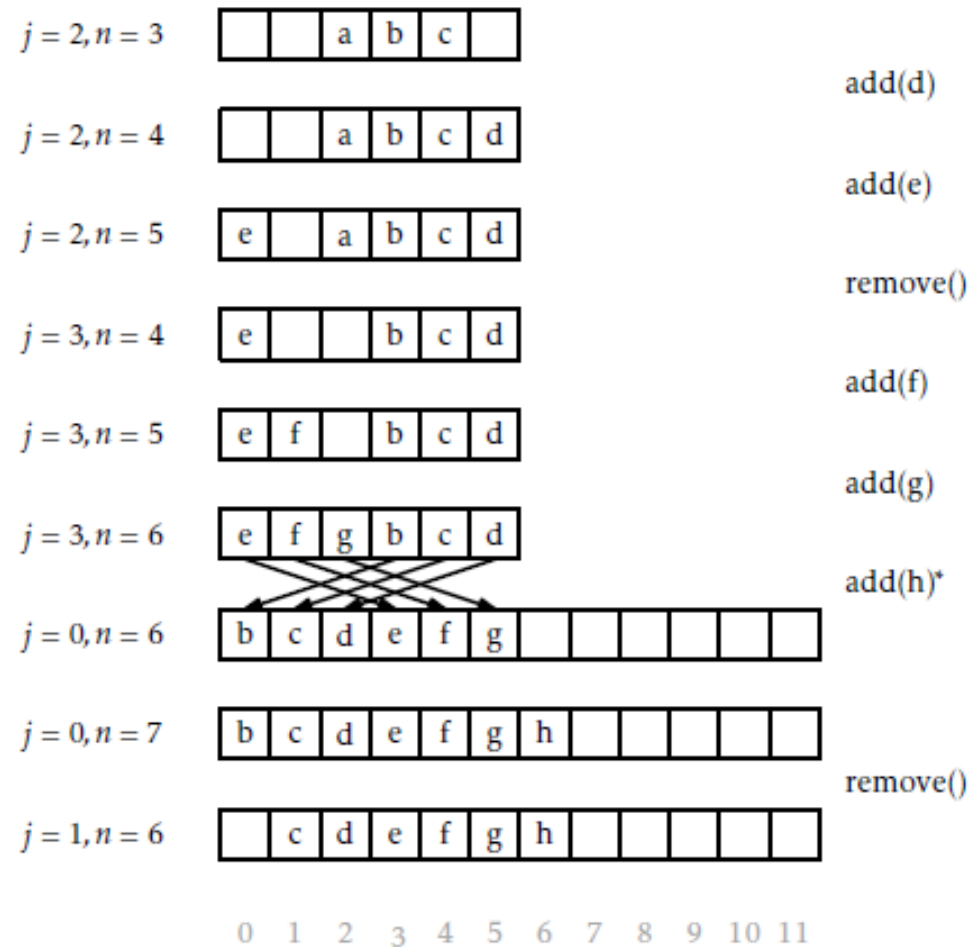


Figure 2.2: A sequence of $\text{add}(x)$ and $\text{remove}(i)$ operations on an ArrayQueue.

ArrayQueue

```
initialize()
```

```
   $a \leftarrow \text{new\_array}(1)$ 
```

```
   $j \leftarrow 0$ 
```

```
   $n \leftarrow 0$ 
```

```
get( $i$ )
```

```
  return  $a[(i + j) \bmod \text{length}(a)]$ 
```

```
set( $i, x$ )
```

```
   $y \leftarrow a[(i + j) \bmod \text{length}(a)]$ 
```

```
   $a[(i + j) \bmod \text{length}(a)] \leftarrow x$ 
```

```
  return  $y$ 
```

```
remove()
```

```
   $x \leftarrow a[j]$ 
```

```
   $j \leftarrow (j + 1) \bmod \text{length}(a)$ 
```

```
   $n \leftarrow n - 1$ 
```

```
  if  $\text{length}(a) \geq 3 \cdot n$  then resize()
```

```
  return  $x$ 
```

ArrayDeque

Adding an element in ArrayDeque

```
add(i, x)  
  if n = length(a) then resize()  
  if i < n/2 then  
    j ← (j - 1) mod length(a)  
    for k in 0, 1, 2, ..., i - 1 do  
      a[(j + k) mod length(a)] ← a[(j + k + 1) mod length(a)]  
  else  
    for k in n, n - 1, n - 2, ..., i + 1 do  
      a[(j + k) mod length(a)] ← a[(j + k - 1) mod length(a)]  
  a[(j + i) mod length(a)] ← x  
  n ← n + 1
```

Removing an element from ArrayDeque

```
remove(i)
   $x \leftarrow a[(j + i) \bmod \text{length}(a)]$ 
  if  $i < n/2$  then
    for  $k$  in  $i, i - 1, i - 2, \dots, 1$  do
       $a[(j + k) \bmod \text{length}(a)] \leftarrow a[(j + k - 1) \bmod \text{length}(a)]$ 
     $j \leftarrow (j + 1) \bmod \text{length}(a)$ 
  else
    for  $k$  in  $i, i + 1, i + 2, \dots, n - 2$  do
       $a[(j + k) \bmod \text{length}(a)] \leftarrow a[(j + k + 1) \bmod \text{length}(a)]$ 
   $n \leftarrow n - 1$ 
  if  $\text{length}(a) \geq 3 \cdot n$  then resize()
  return  $x$ 
```

Complexity

Theorem 2.3. *An ArrayDeque implements the List interface. Ignoring the cost of calls to `resize()`, an ArrayDeque supports the operations*

- *`get(i)` and `set(i, x)` in $O(1)$ time per operation; and*
- *`add(i, x)` and `remove(i)` in $O(1 + \min\{i, n - i\})$ time per operation.*

Fast Array DEQUEUE - Exercise

- Given the DEQUEUE below:



- Perform the following list operations:
 - remove(2)
 - add(4,x)
 - add(3,y)
 - add(4,z)

add_all(i,c) - Discussion

- The List method `add_all(l, c)` inserts all elements of the Collection `c` into the list at position `i`. This can be implemented by repeated calls to `add(i; x)`.
 - *What issue do you see in this?*
 - *What can be more efficient implementation?*

Next Week

- Quiz
- Resize() operation and its analysis

Resources

- Open Data Structures (pseudocode edition), by Pat Morin. Available online at <http://opendatastructures.org>
- Data Structures and Algorithms in Python, by Michael T. Goodrich, Roberto Tamassia, and Michael H. Goldwasser. 2013. (1st. ed.). Wiley Publishing
- <https://www.khanacademy.org/computing/computer-science/algorithms/asymptotic-notation/a/asymptotic-notation>

Thanks